Surgical Tool Tracking on the Magic Leap 2

Final Report - Undergraduate Thesis

Yusuf Nissar

CS 4490Z: Thesis

Department of Computer Science

Western University

March 30, 2026

Project Supervisor: Elvis Chen, Dept. of Electrical and Computer Engineering

Course Coordinator: Nazim Madhavji, Dept. of Computer Science

Glossary:

Head-mounted display (HMD): a wearable device used for augmented or virtual reality.

Augmented reality (AR): a technology that overlays virtual objects or content over a user's view of the real world.

HoloLens 2 (HL2): Microsoft's augmented reality head-mounted display. It runs on Windows and provides a research mode API for accessing sensor data. It was discontinued in 2024.

Magic Leap 2 (ML2): an augmented reality head-mounted display running on Android OS. It is the target platform for this thesis project.

Application programming interface (API): an interface for software to communicate with each other through a predefined group of functions.

Native Development Kit (NDK): A compilation tool for compiling native C and C++ programs into the machine code required by processor chipsets used for Android, enabling high-performance plugin development for the ML2.

Six degrees of freedom (6DoF): the full range of an object's possible movement in 3D space, including three translational axes (x, y, z) and three rotational axes (pitch, yaw, roll).

Retro-reflective marker (RRM): markers that reflect infrared light directly back to the camera sensor, appearing as bright blobs in infrared images and allowing them to be easily detected using computer vision algorithms.

Time of Flight (ToF): a depth-sensing technique that measures the time it takes for emitted light to travel to a surface of an object and return to the sensor. This thesis relies on the ML2's ToF depth sensor.

Surgical Tool Tracking using off-the-shelf Augmented Reality head-mounted displays (STTAR): a framework introduced by Martin-Gomez et al. (2024) that uses time of flight depth sensors on the HoloLens 2 to track retro-reflective markers in 3D space for surgical tool tracking. Although they developed their solution for the HoloLens 2, it can, in principle, be applied to other headsets with similar depth-sensing capabilities.

Field of View (FOV): The angle that describes how much of the world can be seen.

Structured Abstract:

Context and Motivation: The AI-Assisted Intervention Lab uses the HoloLens 2 (HL2) for surgical tool-tracking via a Unity plugin that uses retro-reflective markers (RRMs) for getting a tool's pose. Following the HL2's discontinuation in 2024, the lab hopes to port projects to the Magic Leap 2 (ML2). This research focuses on creating an equivalent tool-tracking Unity plugin for the ML2 to enable these ports and the creation of new tool-tracking-related projects.

Research Question/Objectives: Verify the feasibility of creating a ML2 tool-tracking Unity plugin (across different hardware, OS, and API) and develop a prototype.

Principal Concepts: Key questions include: 1) Whether the ML2's hardware and API expose the necessary data compared to the HL2. 2) How Android-based plugins differ from the HL2's Windows-based plugins. 3) How the STTAR framework uses RRMs to determine tool position and orientation. 4) What ML2-specific issues must be resolved to achieve equivalent functionality.

***Research Methodology*:** This thesis combined exploratory research into the ML2's capabilities with iterative prototype development. The project was broken into subgoals, with the roadmap adjusted based on outcomes at each stage.

***Result of Your Research*:** A ML2 C++ prototype utilizing the STTAR framework to capture the translation and orientation of tools using RRMs.

***Novelty*:** This thesis is the first known effort to implement retro-reflective marker-based surgical tool-tracking on the ML2.

***Impact of Results*:** This thesis's prototype demonstrates the feasibility of tool-tracking on the ML2. It also provides a foundation for a thoroughly tested plugin that will unblock the lab's development on this platform.

Progress to Date: Set up the ML2, verified hardware/API sufficiency, built a Unity plugin for Android, implemented native RRM detection, and added tool recognition and tracking.

***Future Work*:** Improving tracking accuracy by accounting for potential distortions, transferring tracking data into Unity, and resolving potential discrepancies between virtual object movement and real-world motion, verifying tracking accuracy against medical-grade alternatives (NDI Polaris Spectra)

Table of Contents

1. **Introduction**
    - 1.1 General Background and Problem Description
    - 1.2 Focus of This Thesis
    - 1.3 Key Research Questions
    - 1.4 Key Results
    - 1.5 Novelty and Significance
    - 1.6 Report Structure
2. **Background and Related Work**
    - 2.1 Augmented Reality in Surgical Navigation
    - 2.2 Surgical Tool Tracking Approaches
    - 2.3 The STTAR Framework
    - 2.4 The Magic Leap 2 as an AR Platform
    - 2.5 Analysis and Research Gap
3. **Research Objectives**
4. **Methodology**
    - 4.1 Development Approach
    - 4.2 Depth Data Acquisition
    - 4.3 STTAR Framework Implementation
    - 4.4 Tools and Libraries
5. **Results**
    - 5.1 System Overview and Architecture
    - 5.2 Platform Exploration Findings
    - 5.3 Marker Detection Pipeline
    - 5.4 Tool Recognition via Graph Search
    - 5.5 Pose Estimation and Filtering
    - 5.6 System Validation and Testing
    - 5.7 ML2-Specific Challenges
    - 5.8 Novelty of Results
6. **Discussion**
    - 6.1 Threats to Validity
    - 6.2 Implications
    - 6.3 Limitations
    - 6.4 Generalisability
7. **Conclusion**
8. **Future Work and Lessons Learnt**
    - 8.1 Future Work
    - 8.2 Lessons Learnt
9. **Acknowledgements**
10. **Citations**

1. Introduction

1.1 General Background and Problem Description

Many of the AI-Assisted Medical Intervention Lab's projects use the HoloLens 2 (HL2), an augmented reality (AR) head-mounted display (HMD) for surgical tool-tracking. These projects utilize a Unity plugin to track the position and orientation of different tools by attaching retro-reflective markers (RRMs) to them. This plugin, developed by Keller (2023), is based on the STTAR (Surgical Tool Tracking using Augmented Reality) framework introduced by Martin-Gomez et al. (2024), which relies on the HMD's time-of-flight (ToF) depth camera to detect RRMs and compute tool pose. However, after the HL2 was discontinued in 2024, the lab has been looking to port their projects over to another AR HMD, the Magic Leap 2 (ML2). Without a similar tool-tracking Unity plugin, porting projects has not been feasible. Compounding this issue is a general lack of development resources targeting the ML2 for this purpose, as well as the lab's limited familiarity with the headset and its capabilities. Creating a comparable plugin would remove a major roadblock in the lab's transition to the new platform.

1.2 Focus of This Thesis

This thesis explores the capabilities of the ML2 and develops a prototype that can track surgical tools fitted with retro-reflective markers. Because the lab has limited prior experience with the ML2, the project is partly exploratory in that I will also be assessing whether a plugin following the STTAR framework is feasible on this platform. The work proceeded in two phases. First, it was necessary to understand how to set up and develop for the ML2, including differences from the HL2 in operating system (Android versus Windows), available APIs, and how Unity interacts with the underlying platform. Second, the STTAR framework had to be implemented on the ML2's hardware, covering the marker identification process, tool recognition logic, and tracking pipeline. This was not a straightforward port as the STTAR framework depends on the HMD's ToF depth camera, and the headset’s respective cameras and hardware differ in many ways. For example, both cameras have different resolutions.

1.3 Key Research Questions

This thesis addresses four questions. First, does the ML2's hardware and API expose the necessary depth data information required, such as raw depth images and infrared intensity values at a high frame-rate, similar to the HL2? Second, how do Android-based native plugins differ from the HL2's Windows-based plugins in terms of build systems and lifecycle management? Third, how does the STTAR framework use RRMs to determine tool position and orientation? Fourth, what ML2-specific issues, such as depth image quality and spatial distortions, must be resolved to achieve equivalent functionality?

1.4 Key Results

The project provided two main outcomes. Firstly, an exploration of the ML2's depth-sensing capabilities confirmed that the headset provides raw depth and infrared intensity data, with a short-range mode providing a fast frame rate suitable for real-time tracking. Secondly, a C++ prototype was developed that implements the STTAR framework to capture the translation and orientation of tools using RRMs. The original objective was to deliver a complete Unity plugin;

however, technical challenges involving ML2 setup and handling ML2-specific issues, as well as resource constraints considering the lab only had one ML2 headset, meant that additional time would be required to reach a fully integrated Unity solution. The prototype nonetheless demonstrates the viability of the approach and lays the groundwork for future plugin development that should involve thorough testing before use.

1.5 Novelty and Significance

This thesis represents the first known effort to implement RRM-based surgical tool tracking on the ML2. The findings confirm that the ML2 is a viable successor platform for the lab and provide a concrete prototype from which a production-ready plugin can be developed. Other researchers have recently begun pursuing similar projects, underscoring the broader demand for tool-tracking capabilities on the ML2.

1.6 Report Structure

The remainder of this report is organized as follows. Section 2 presents background and related work. Section 3 outlines the research objectives. Section 4 describes the research methodology, covering the technology used and development process on a high-level. Section 5 describes the solution and its results. Section 6 discusses the weaknesses of the findings, limitations of the results, implications of the research and its generalizability. Section 7 acts as a conclusion to the thesis. Section 8 focuses on the future work necessary and lessons learned. Section 9 concludes the report with acknowledgements of others who helped in this research.

2. Background and Related Work

2.1 Augmented Reality in Surgical Navigation

Augmented reality has seen a wide variety of applications in several surgical fields. In spine surgery, AR-assisted pedicle screw placement has shown potential for improving accuracy and reducing reliance on fluoroscopy. Gibby et al. (2018) demonstrated that a Microsoft HoloLens with superimposed CT data could guide pedicle screw placement on a phantom model without real-time fluoroscopic guidance, while Viehöfer et al. (2020) showed that HoloLens-guided osteotomies improved accuracy for less experienced surgeons. However, both studies were conducted under simplified experimental conditions using static phantom or bone models and with only a few participants. A major weakness of existing AR intervention studies is that they rely on a manual tool registration process, where the surgeon aligns the virtual hologram to visual landmarks. This is both a highly subjective and time-consuming process that makes it difficult to justify the real-world usage of these tools. This weakness is reflected in how surgical-grade AR navigation systems use infrared cameras and RRMs for tool recognition and tracking, enabling the precise accuracy required in the surgical room.

2.2 Surgical Tool Tracking Approaches

The conventional approach to surgical tool tracking relies on external medical-grade infrared camera systems that detect RRMs attached to tools and compute their six-degree-of-freedom (6-DoF) pose (NDI, 2024). These systems are accurate but expensive and add hardware complexity to the operating room. However, commercially available AR HMDs also contain infrared cameras for self-localization and hand tracking, which has motivated researchers to explore whether these sensors could be used for tool tracking.

Kunz et al. (2020) investigated infrared marker tracking on the HoloLens for neurosurgical interventions using two approaches: one combining the ToF sensor's reflectivity and depth information, and another using the front-facing grayscale cameras as a stereo pair. The depth-based approach showed a tracking accuracy of 0.76 mm, but only between 40 and 60 cm and without considering rotation. The stereo approach required additional illumination to make markers visible, and the authors did not report its tracking accuracy. Gsaxner et al. (2021) combined a stereo vision pipeline for initial marker detection with a Kalman filter for continuous 6-DoF tracking, achieving an accuracy of 1.7 mm and 1.11°. However, both approaches still require external infrared illumination of the RRMs, which acts as a hardware constraint and can slow adoption from surgical-grade systems to cheaper HMD-based alternatives. However, it is important to note that both of these used the HoloLens 1, which may have been the limiting factor.

2.3 The STTAR Framework

Martin-Gomez et al. (2024) created the STTAR framework, which relied exclusively on the HoloLens 2's ToF sensor operating at 45 fps with a 127° × 127° field of view (FOV), to detect RRMs without requiring external illumination.

The framework operates in three stages. Firstly, marker centroid detection using intensity thresholding and back-projection into 3D coordinates in camera space via the depth data and

camera intrinsics. Secondly, the tool definition records pairwise distances between markers across multiple frames to build a stable and unique fingerprint of distances for each tool. Lastly, tool tracking, which matches observed pairwise inter-marker distances against stored tool definitions.

Per-marker Kalman filters improve precision by exploiting the fact that depth error follows a normal distribution with standard deviation increasing with distance. The system runs at 172 Hz for one tool and 51 Hz for five tools, with approximately 103 ms latency relative to an NDI reference tracker. As a part of this research, Keller (2023) implemented STTAR as a Unity plugin for HL2, which became the foundation for multiple lab projects.

2.4 The Magic Leap 2 as an AR Platform

The ML2 is an Android-based AR HMD with a different hardware and software architecture compared to the Windows-based HL2. Both provide depth and infrared intensity data with a short-range mode enabling higher frame rates. However, its ToF depth sensor has several key differences. Its native resolution of 640 x 240 px is cropped and interpolated into a resolution of 544 x 480 px with a FOV of 75° × 70° compared to the HL2's AHAT (short-range ToF) camera (512 × 512, 127° × 127° FOV). The depth precision is worse for the ML2 as well as its modulation frequency is a maximum of 100 MHz (Infineon, 2019) whereas the HL2 uses the Azure Kinect DK, which has a modulation frequency of 200 to 320 MHz (Kurillo et al., 2022). Its Android-based development environment also requires native plugins to be built as shared libraries (.so) files rather than the dynamic-link libraries (.dll) used on the HL2's Windows platform.

2.5 Analysis and Research Gap

Research in HMD tracking approaches by Kunz et al. (2021) and Gsaxner et al. (2021) demonstrated that early AR headset sensors could track RRMs but required supplementary infrared illumination, partially negating the convenience of commercially available headsets. The STTAR framework (Martin-Gomez et al., 2024) resolved this by solely leveraging the HL2's ToF sensor and achieving clinically comparable accuracy. Keller's (2023) Unity plugin made this practically accessible for development, and it has been what the lab has used thus far.

However, all existing STTAR implementations are tied to the HoloLens 2. With the HL2's discontinuation, this work risks becoming inaccessible to researchers and surgeons who lack the hardware. The ML2 offers a viable alternative with a ToF sensor capable of providing the necessary data, but no prior attempts have been made to implement a similar tracking system on it. This thesis addresses that gap by exploring the ML2's depth-sensing capabilities and developing a prototype STTAR implementation on the headset.

3. Research Objectives

The objectives of this project are as follows:

- O1: Investigating the Magic Leap 2's depth-sensing hardware and API to determine whether it exposes the necessary data required to implement the STTAR framework. Requirements include: access to raw depth information streams at a high frame rate and camera intrinsics.
- O2: Understanding the differences between developing native plugins for the ML2's Android-based platform versus the HL2's Windows-based platform, including build systems (.so versus .dll) and Unity integration.
- O3: Implementing the STTAR framework's three-stage pipeline which includes marker centroid detection, tool definition, and tool tracking on the Magic Leap 2's hardware.
- O4: Identifying and documenting ML2-specific issues, such as depth image quality and spatial distortions, that must be resolved to achieve tracking functionality equivalent to the HL2 implementation.
- O5: Delivering a working C++ prototype that captures the translation and orientation of surgical tools using retro-reflective markers on the ML2, laying the foundation for a future Unity plugin.

4. Methodology

This project followed a two-phase research and development approach. The first phase focused on exploring the ML2's setup process, development environment and depth-sensing capabilities. The second phase implemented the STTAR framework's three-stage tracking pipeline on the ML2 hardware.

4.1 Development Approach

The prototype was built as a native Android application in C++ rather than as a Unity plugin. This decision was driven by the need to build the plugin natively and connect it to Unity via a shared library once effective tool tracking has been achieved. Additionally, this approach allowed me to investigate whether the ML2's depth camera could provide usable data for RRM-based tracking before investing in creating a Unity integration. Using an existing Magic Leap C++ sample that provided an ImGui-based interface, I was able to create a C++ prototype that allowed for real-time parameter tuning using the Magic Leap C API (CAPI), enabling rapid iteration. The Android Native Development Kit (NDK) was used with CMake and Gradle to compile the application, with a minimum SDK version of 29 as required by the ML2 documentation.

4.2 Depth Data Acquisition

Depth data was accessed through the ML2's C API depth camera module, which provides per-frame buffers for depth, raw infrared intensity, ambient infrared intensity, and depth confidence. Camera intrinsics are also provided, such as focal length, principal point, and distortion coefficients. The camera was configured in short-range mode to maximize frame rate for real-time tracking. Head pose was obtained through the ML2's C API Head Tracking and Perception module, providing the camera-to-world transform at each frame.

4.3 STTAR Framework Implementation

The STTAR framework (Martin-Gomez et al., 2024) was implemented following its three-stage pipeline: marker detection, tool recognition, and pose estimation. Marker detection uses the infrared intensity image to identify RRM candidates through image processing techniques and validates them against the expected marker size at a given depth. Tool recognition matches detected markers to registered tool templates using pairwise distances between markers. Pose estimation computes the 6-DoF tool pose using the Kabsch algorithm to find the optimal rigid-body alignment between template and detected marker positions. Kalman filtering and temporal smoothing were applied to reduce per-frame jitter.

4.4 Tools and Libraries

OpenCV 4.12 was used across the project. The Magic Leap SDK provided the necessary depth camera, head tracking, and perception data. The prototype outputs the same 8×1 pose vector format as the HL2 plugin ([x, y, z, qx, qy, qz, qw, valid]) to facilitate future Unity integration.

5. Results

5.1 System Overview and Architecture

The C++ prototype created as a part of this thesis implements the STTAR framework and runs natively on the ML2. Each frame is processed in 4 steps: depth buffer data is acquired through the ML2's C API, markers are detected through infrared intensity thresholds and 3D back-projected, tool recognition happens through processing several frames of a tool's inter-marker distances and representing the tool via a sorted list of these distances, and tool tracking using the Kabsch algorithm with Kalman filtering and temporal smoothing to reduce jitteriness.

The final output is an 8×1 pose vector [x, y, z, qx, qy, qz, qw, valid] which is the same format used by Keller's (2023) HL2 Unity plugin. This allows existing Unity projects developed for the HL2 to consume the same tool poses without needing any modification when a future plugin is developed.

5.2 Platform Exploration Findings

This section addresses objectives O1 and O2.

Operating system and development environment. The HL2 runs Windows and builds native plugins as UWP apps using Visual Studio, producing a .dll and a .winmd file. The .winmd file exposes the plugin's API to Unity's C# layer via WinRT marshalling, with the plugin functions accessible within Unity defined in a .idl file. The ML2 runs Android and uses the NDK with CMake and Gradle, producing a .so file. Unity integration on Android uses JNI or direct native plugin loading via DllImport, with no .winmd or .idl equivalent. Table 1 summarises the key platform differences.

Aspect	HoloLens 2	Magic Leap 2
Operating system	Windows 10 (UWP)	Android
Build system	Visual Studio .vcxproj	CMake + Gradle
Build output	.dll + .winmd	.so
Unity interface	.idl → WinRT marshalling	JNI or DllImport
Linear algebra	DirectXMath (SIMD)	OpenCV cv::Mat

Table 1: Platform Differences Between Android and Windows Plugins

Sensor access APIs. On the HL2, accessing the depth sensor requires enabling Research Mode, a special API that is dynamically loaded at runtime. The AHAT sensor provides depth as UINT16 in millimetres and an Active Brightness (AB) infrared image as UINT16. On the ML2, the Depth Camera API is a standard C API requiring no special mode. It provides four buffers per-frame, depth in meters, raw infrared intensity, ambient infrared intensity, and depth

confidence. Additionally, it provides camera intrinsics, including five distortion coefficients. The camera-to-world transform is provided directly per-frame as well.

Depth and infrared data confirmation (O1). The ML2's short-range mode provides all data streams necessary for marker-based tracking. The infrared intensity image clearly shows retro-reflective markers as bright spots without external illumination, enabling the same marker detection process used by STTAR on the HL2.

5.3 Marker Detection

Marker detection is done by processing the infrared intensity image in each frame to identify RRMs and recover their 3D positions. When enabled, ambient infrared intensity can be removed to improve the signal-to-noise ratio, although further testing needs to be done to verify whether this results in a meaningful improvement in all scenarios. A Gaussian blur (default 5 × 5) smooths scattered bright pixels into coherent blobs, followed by intensity thresholding (default range 2000–3500, determined empirically) to isolate the RRM. Morphological closing with an elliptical structuring element merges nearby components, and OpenCV's connectedComponentsWithStats identifies individual blobs with their centroids and areas.

Each blob is validated against three criteria: the centroid must fall within image bounds, the depth value must be valid, and the blob area must fall within 0.5x–2.0x of the expected area, computed as $A_px \approx \pi \cdot r^2 \cdot fx \cdot fy / d^2$, where r is the marker radius, fx and fy are the focal lengths, and d is the depth. Accepted centroids are corrected for lens distortion using OpenCV's undistortPoints with the ML2's five-parameter model, then back-projected into 3D camera-space coordinates using the depth value corrected for the marker's physical radius. Figure 1 shows the infrared intensity image with detected markers highlighted as coloured overlays.

Figure 1: Marker Detection

Correctly identified markers are displayed as red blobs on the depth camera

5.4 Tool Recognition via Graph Search

Each tool is defined by a set of sphere positions in a local coordinate frame. During registration, the user must hold the tool in view while the system collects marker positions over multiple frames, and a minimum of 3 markers must be detected. These marker positions are then Kabsch aligned to a reference frame, and then an average is derived from all of the aligned frames. It then realigns the collected frames to this average, and the process repeats to converge onto a pairwise distance matrix. This matrix is then stored and treated as a template for the tool.

Recognition works by searching for groups of detected markers whose pairwise distances match a stored tool template. The search begins by finding a "seed edge", a pair of detected markers whose distance matches a template edge within tolerance. This tolerance is included to account for depth-related noise that may cause pairwise distances to appear longer or shorter than they truly are and is ultimately the larger of a fixed floor (default 4 mm) or a relative fraction (default 5%) of the expected distance.

From each seed, the search expands depth-first, adding one candidate sphere at a time. Each new candidate is validated by checking that its distances to all previously assigned spheres match the corresponding template distances. To handle partial visibility, a configurable number of template nodes may be marked as occluded and skipped.

A candidate match is accepted when the mean per-edge error falls below the adaptive threshold. When multiple tools are registered, candidates are ranked by quality. Tools identified with the fewest occlusions first, then lowest total error, and assigned greedily, discarding any candidate that reuses a sphere already claimed by a higher-ranked match.

5.5 Pose Estimation and Filtering

Before computing the tool's pose, each detected sphere's 3D position is smoothed by a per-sphere Kalman filter (constant-velocity model, six states: position and velocity in three axes) to reduce frame-to-frame jitter.

The smoothed positions are then aligned to the tool template using the Kabsch algorithm (SVD with a determinant check to prevent reflections) if a minimum of 3 markers are detected, producing a rigid-body transform in camera space. This is converted to world space using the ML2's camera-to-world transform.

Finally, a temporal low-pass filter smooths the output pose: linear interpolation for position ($\alpha = 0.6$) and SLERP for orientation ($\beta = 0.3$). The rotation quaternion is extracted using Shepperd's method. The final output is an 8×1 pose vector [x, y, z, qx, qy, qz, qw, valid].

5.6 System Validation and Testing

Because the prototype depends on physical hardware, validation relies on testing via an ImGui interface displayed to the user, device logging through Android logcat, and manual experiments.

Experiments were conducted using four RRMs mounted onto a rigid tool body. The tool was held at varying distances (approximately 30, 60, 90, and 120 cm) and at multiple orientations

(horizontal, vertical, and oblique). At each configuration, marker detection, tool recognition, and tool tracking were evaluated. The results showed reliable detection, recognition, and tracking at close range (0.3–0.5 m) across all orientations, confirming that the pairwise-distance approach is orientation-invariant. At greater distances in all axes, marker detection continued to function, but tool recognition can fail due to pairwise distance errors exceeding the graph-search tolerance. This distance-dependent degradation was consistent across all orientations and is discussed in Section 5.7.

5.7 ML2-Specific Challenges

This section addresses objective O4. While the prototype demonstrates that the STTAR framework is viable on the ML2 with additional tuning and testing, several issues present challenges that do not exist on the HL2.

Narrower field of view. The ML2's comparatively smaller FOV of 75° × 70° compared to the HL2's 127° × 127° significantly reduces the space in which markers can be detected, requiring tools to remain more centrally positioned in the user's gaze.

Non-linear depth scale error. Pairwise inter-marker distances computed from back-projected 3D positions should remain constant regardless of depth. Instead, logged data showed inconsistent distortion: distances roughly doubled from 30 cm to 60 cm, then dropped back toward 30 cm values at 90 cm and 120 cm. Repeated sessions at the same distance also produced different distance sets due to orientation-dependent errors. This non-linear, orientation-dependent bias means templates registered at one depth and angle will not reliably match at other working distances, and a single relative tolerance cannot fully compensate. Further research needs to be done to understand how to correct for this error in a more reliable manner than having large tolerances for pairwise distance errors in tool tracking.

Suspected x-y plane distortion. Visual inspection of infrared images suggests spatial distortion not fully corrected by the applied five-parameter distortion model. Markers near the edges of the field of view appear displaced relative to centre markers, producing asymmetric pairwise distance errors. This has not yet been validated through controlled calibration and remains a hypothesis.

Infrared intensity noise. The ML2's raw infrared image exhibits significant noise, with intensity image values for markers appearing scattered rather than as clean bright blobs. To counteract this, Gaussian blurring and morphological closing are used to produce more coherent intensity images. These preprocessing steps were not necessary for the HL2’s depth sensor. This may be attributable to the ML2’s lower modulation frequency of a maximum of 100 MHz (Infineon, 2019) compared to the HL2’s Azure Kinect DK’s modulation frequency of 200 to 320 MHz (Kurillo et al., 2022).

5.8 Novelty of Results

This thesis is the first implementation of marker-based tool tracking on the ML2. Prior implementations of the STTAR framework (Martin-Gomez et al., 2024; Keller, 2023) are exclusive to the HL2. This prototype also confirms that the ML2's ToF sensor provides sufficient infrared illumination for passive marker detection, matching the key advantage STTAR achieved

on the HL2. The ML2-specific challenges in Section 5.7 are also novel findings that any future tracking implementation on this platform will need to address.

The prototype's output format was deliberately matched to Keller's (2023) HL2 plugin interface, providing the lab with a path forward to creating a full Unity plugin without requiring future developers to repeat the exploratory work carried out in this thesis.

6. Discussion

6.1 Threats to Validity

All testing was conducted using 1 tool with a 4 RRM rigid body attached to it in a well-lit indoor environment without strict testing practices. Performance validation was primarily for the purpose of confirming tracking success or failure and tool pose stability. Therefore, further testing needs to take place to verify the real-world translation and rotation accuracy of the ML2. Additionally, this testing needs to take place in a variety of environments with multiple tools, as I only tested with one tool, with the system theoretically working with multiple. Additionally, as identified in 5.7, distortions and depth scaling errors need further analysis to ensure that the final plugin works effectively in all environments.

6.2 Implications

This thesis has proven that the ML2 is a viable platform for future tool-tracking projects and has provided a strong foundation for the lab to easily create a production-ready Unity tool-tracking plugin. The completion of a plugin would also be useful to other labs across the world conducting similar research, as indicated by the other similar projects taking place at other universities.

By using the STTAR framework to build this project, it also proves that the framework is generalizable to other AR HMDs that have ToF sensing capabilities, with some adjustments based on the capabilities of the headset. This finding is extremely important as it provides a clear path forward for other researchers in the case that the ML2 is discontinued as well.

6.3 Limitations

Currently, tracking is reliable only at close range (0.3–0.5 m) due to non-linear depth scale error and increasing noise, causing failures in tool detection. The prototype also remains a standalone C++ application, rather than a Unity plugin.

6.4 Generalisability

The findings related to depth sensor errors generalize well to other ML2 projects heavily reliant on the ToF sensor for any form of tracking. Additionally, proving that the STTAR framework generalizes to other AR HMDs is a finding that gives more credibility to the framework and its usefulness. However, the results of the thesis itself are specific to the ML2 and may not generalize well to other AR HMDs, which may have their own hardware, OS and depth sensing issues.

7. Conclusion

The discontinuation of the HL2 left the AI-Assisted Medical Intervention Lab without a supported platform for its RRM-based surgical tool tracking, which relied on the STTAR framework (Martin-Gomez et al., 2024) and its Unity plugin (Keller, 2023). This thesis explored the Magic Leap 2 as a successor platform, implemented the STTAR pipeline on its hardware, identified ML2-specific challenges, and delivered a working prototype.

All five objectives were met. The platform exploration (Section 5.2) confirmed that the ML2's ToF sensor provides the necessary depth and infrared data for RRM detection (O1) and documented key differences between the HL2 and ML2 development environments (O2). A C++ prototype implements the full STTAR pipeline: marker detection, tool recognition, and tool tracking, achieving reliable close-range tracking across multiple tool orientations (O3, O5). The ML2-specific challenges documented in Section 5.7, including non-linear depth scale error, suspected x-y distortion, and infrared noise, represent novel findings not previously reported in ML2 literature (O4).

The primary conclusion of this thesis is that the ML2 is viable for STTAR-based tool tracking, but further research needs to be done into correcting for ML2-specific errors and verifying its accuracy against other infrared tracking devices, such as the NDI Polaris. The prototype and this thesis’s findings will provide the lab with a strong foundation to continue this work and build a tool-tracking Unity plugin.

8. Future Work and Lessons Learnt

8.1 Future Work

Unity plugin integration. The next step for this project would be to expose the prototype's tracking output in Unity and verify that it functions as intended. The output format already matches Keller's (2023) HL2 plugin, so Unity-side consuming code should require minimal modification.

Depth sensor calibration. The non-linear depth scale error (Section 5.7) is the primary barrier to tracking beyond close range. Additional testing needs to be done to understand why the ML2's intrinsic parameters do not already solve for this and what additional fixes need to be added. Currently, large thresholds for pairwise distance errors are being used, but this is not a sustainable solution in the long-run as it does not address the problem directly.

Display calibration and the transformation chain. Rendering tracked tool poses in AR requires transforming positions from camera space to the user's eye position. The current prototype transforms from camera to world space, but Unity rendering will require an additional transformation step to account for the depth camera-to-eye offset. The ML2 reports automatic correction for eye position, but how this is exposed to developers and whether it is accurate remains unclear.

Quantitative evaluation. Validation should also be repeated with comparisons to surgical-grade AR navigation systems such as the NDI Polaris (e.g., NDI) to produce accuracy measurements similar to Martin-Gomez et al. (2024).

8.2 Lessons Learnt

ToF sensor behaviour cannot be assumed to be consistent across devices. The HL2's depth error follows a predictable normal distribution that can be modelled with Kalman filters (Martin-Gomez et al., 2024). The ML2 exhibits non-linear errors that vary with both distance and orientation (Section 5.7). Other researchers porting depth-dependent algorithms between AR devices should consider this rather than assuming that it can be directly transferred.

Platform documentation gaps require exploratory prototyping. The ML2's SDK documentation does not clearly document the depth sensor's error behaviour. Additionally, the ML2's OpenXR API does not provide access to raw depth data streams, indicating a lack of focus on development for these types of projects. For emerging AR platforms with limited third-party research, building a minimal exploratory prototype before committing to a full implementation is more effective than relying on documentation alone.

Output interface compatibility reduces migration cost. Matching the existing HL2 plugin's output format decouples the tracking backend from the consuming application, allowing downstream Unity code to remain unchanged and reducing migration scope to platform-dependent components only.

9. Acknowledgements

I would like to express my gratitude to Professor Elvis Chen and Daniel Allen, a postdoctoral researcher in the lab, for providing me with this unique learning opportunity as well as for their guidance and encouragement over the past few months. I am deeply appreciative of this opportunity to work on this project as well as hear their insights during the project. Their participation has been crucial in helping to shape this research's final deliverables.

10. Citations

Gibby, J. T., Swenson, S. A., Cvetko, S., Rao, R., & Javan, R. (2018). Head-mounted display augmented reality to guide pedicle screw pflacement utilizing computed tomography. *International Journal of Computer Assisted Radiology and Surgery*, *14*(3), 525–535. https://doi.org/10.1007/s11548-018-1814-7

Gsaxner, C., Li, J., Pepe, A., Schmalstieg, D., & Egger, J. (2021). Inside-out instrument tracking for surgical navigation in augmented reality. *Proceedings of the 27th ACM Symposium on Virtual Reality Software and Technology*, 1–11. https://doi.org/10.1145/3489849.3489863

Infineon. (2019). Product brief real3 image sensor irs2877a(s). https://www.infineon.com/dgdl/Infineon-IRS2877AS_REAL3_image_sensor-ProductBrief-v01_00-EN.pdf?fileId=8ac78c8c80f4d32901814971dc03260c

Kunz, C., Maurer, P., Kees, F., Henrich, P., Marzi, C., Hlaváč, M., Schneider, M., & Mathis-Ullrich, F. (2020). Infrared marker tracking with the Hololens for neurosurgical interventions. *Current Directions in Biomedical Engineering*, *6*(1). https://doi.org/10.1515/cdbme-2020-0027

Kurillo, G., Hemingway, E., Cheng, M.-L., & Cheng, L. (2022b, March 23). *Evaluating the accuracy of the Azure Kinect and Kinect v2*. Sensors (Basel, Switzerland). https://pmc.ncbi.nlm.nih.gov/articles/PMC9002889/

Liebmann, F., Roner, S., von Atzigen, M., Scaramuzza, D., Sutter, R., Snedeker, J., Farshad, M., & Fürnstahl, P. (2019). Pedicle screw navigation using surface digitization on the Microsoft hololens. *International Journal of Computer Assisted Radiology and Surgery*, *14*(7), 1157–1165. https://doi.org/10.1007/s11548-019-01973-7

Optical Navigation Technology / Optical Tracking. NDI. (2024, March 25). https://www.ndigital.com/optical-navigation-technology/

Viehöfer, A. F., Wirth, S. H., Zimmermann, S. M., Jaberg, L., Dennler, C., Fürnstahl, P., & Farshad, M. (2020). Augmented reality guided osteotomy in Hallux Valgus correction. *BMC Musculoskeletal Disorders*, *21*(1). https://doi.org/10.1186/s12891-020-03373-4

Yoon, J. W., Chen, R. E., Han, P. K., Si, P., Freeman, W. D., & Pirris, S. M. (2016). Technical feasibility and safety of an intraoperative head‐up display device during spine instrumentation. *The International Journal of Medical Robotics and Computer Assisted Surgery*, *13*(3). https://doi.org/10.1002/rcs.1770